

# *Linux Inter-Process Communication Using Pipes and FIFO*

## 1. Aim

To implement and demonstrate **Inter-Process Communication (IPC)** in Linux using **Pipes and FIFOs (Named Pipes)**, allowing processes to exchange messages and coordinate their activities.

Your project document identifies the project as “**Linux Inter-Process Communication Using Pipes and FIFO.**”

---

## 2. Abstract

Inter-Process Communication (IPC) allows processes to exchange information and coordinate tasks. This project demonstrates two Linux IPC mechanisms: **pipes** and **FIFOs**.

A pipe is used for communication between related processes, such as a parent and child created using `fork()`. A FIFO, or named pipe, allows unrelated processes to communicate through a special file in the filesystem.

The project uses Linux system calls such as `pipe()`, `mkfifo()`, `read()`, `write()`, and `close()` to establish communication channels and transfer messages between processes.

---

## 3. Objectives

1. To demonstrate communication between Linux processes using **pipes and FIFOs**.
  2. To implement message transfer using `pipe()`, `mkfifo()`, `read()`, and `write()`.
  3. To demonstrate communication between related processes using pipes.
  4. To demonstrate communication between unrelated processes using FIFOs.
  5. To understand process cooperation, synchronization, and resource sharing.
- 

## 4. Software Requirements

| Requirement   | Purpose               |
|---------------|-----------------------|
| Ubuntu/Linux  | Operating environment |
| C Programming | Implementation        |
| GCC           | Compilation           |
| Terminal      | Execution and testing |

These are the tools/platforms specified in your project document.

---

## 5. Linux System Calls Used

| System Call           | Purpose                                                   |
|-----------------------|-----------------------------------------------------------|
| <code>fork()</code>   | Creates a child process                                   |
| <code>pipe()</code>   | Creates a communication channel between related processes |
| <code>mkfifo()</code> | Creates a named pipe                                      |
| <code>read()</code>   | Reads data from the communication channel                 |
| <code>write()</code>  | Sends data through the communication channel              |
| <code>close()</code>  | Closes file descriptors                                   |

---

## 6. Project Methodology

### A. Pipe Communication

1. Create a pipe using `pipe()`.
2. Create a child process using `fork()`.
3. The parent process writes a message into the pipe.
4. The child process reads the message.
5. Both processes close their respective file descriptors.

## B. FIFO Communication

1. Create a named pipe using `mkfifo()`.
2. Run the FIFO reader process.
3. Run the FIFO writer process.
4. The writer sends a message through the FIFO.
5. The reader receives and displays the message.
6. Both processes close the FIFO after communication.

This follows the methodology specified in your project submission.

---

# 7. Project Files

Your project currently contains:

```
pipeline_ipc/  
├── pipe.c  
├── fifo_writer.c  
├── fifo_reader.c  
├──  
├── pipe  
├── fifo_writer  
├── fifo_reader  
├──  
├── myfifo  
└── program.c
```

The **three main source files** are:

- `pipe.c`
  - `fifo_writer.c`
  - `fifo_reader.c`
- 

# 8. Pipe Program

File: `pipe.c`

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int fd[2];
    pid_t pid;

    char message[] = "Hello Child! This message is sent through a Pipe.";
    char buffer[100];

    if (pipe(fd) == -1)
    {
        perror("Pipe creation failed");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid > 0)
    {
        close(fd[0]);

        printf("Parent Process\n");
        printf("Sending message to child...\n");

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]);

        wait(NULL);

        printf("Parent process completed.\n");
    }
    else

```

```
{
    close(fd[1]);

    read(fd[0], buffer, sizeof(buffer));

    printf("Child Process\n");
    printf("Received message: %s\n", buffer);

    close(fd[0]);
}

return 0;
}
```

---

## 9. FIFO Writer Program

**File: `fifo_writer.c`**

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>

#define FIFO_NAME "myfifo"

int main()
{
    int fd;
    char message[100];

    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        /* FIFO may already exist */
    }

    printf("===== FIFO WRITER =====\n");

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
```

```
message[strlen(message, "\n")] = '\0';

fd = open(FIFO_NAME, O_WRONLY);

if (fd == -1)
{
    perror("Error opening FIFO");
    return 1;
}

write(fd, message, strlen(message) + 1);

printf("Message sent successfully through FIFO.\n");

close(fd);

return 0;
}
```

---

## 10. FIFO Reader Program

**File: `fifo_reader.c`**

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_NAME "myfifo"

int main()
{
    int fd;
    char buffer[100];

    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        /* FIFO may already exist */
    }

    printf("===== FIFO READER =====\n");
```

```
fd = open(FIFO_NAME, O_RDONLY);

if (fd == -1)
{
    perror("Error opening FIFO");
    return 1;
}

read(fd, buffer, sizeof(buffer));

printf("Message received through FIFO: %s\n", buffer);

close(fd);

return 0;
}
```

---

## 11. Compilation Commands

Run:

```
gcc pipe.c -o pipe
gcc fifo_writer.c -o fifo_writer
gcc fifo_reader.c -o fifo_reader
```

---

## 12. Pipe Execution

Command:

```
./pipe
```

### Output

Parent Process

Sending message to child...

Child Process

Received message: Hello Child! This message is sent through a Pipe.

Parent process completed.

## Screenshot

 Insert your Pipe terminal screenshot here.

Your actual screenshot demonstrates successful parent-to-child communication.

---

## 13. FIFO Execution

### Terminal 1

```
./fifo_reader
```

### Terminal 2

```
./fifo_writer
```

Enter:

hello

### Output

Writer:

```
===== FIFO WRITER =====
```

```
Enter message: hello
```

```
Message sent successfully through FIFO.
```

Reader:

```
===== FIFO READER =====
```

```
hello
```

## Screenshot

 Insert your FIFO terminal screenshot here.

Your actual terminal screenshot shows that `hello` was successfully transferred through the FIFO.

---

# 14. Testing and Validation

The project was tested using sample messages.

## Test Case 1 — Pipe

### Input/Message:

Hello Child! This message is sent through a Pipe.

### Result:

Child Process

Received message: Hello Child! This message is sent through a Pipe.

**Status:** Successful 

## Test Case 2 — FIFO

### Input:

hello

### Result:

Message received through FIFO: hello

**Status:** Successful 

The project document states that testing should verify message exchange and reliable communication between processes.

---

# 15. Result

The Linux IPC project was successfully implemented using **Pipes and FIFOs**.

- Pipe communication between the parent and child process was successfully demonstrated.
- FIFO communication between independent processes was successfully demonstrated.
- Messages were successfully transferred using Linux system calls.

This satisfies the project's expected outcome of demonstrating communication between related and unrelated processes.

---

## 16. Conclusion

The project provides practical understanding of **Inter-Process Communication in Linux**. Pipes were used for communication between related processes, while FIFOs were used for communication between independent processes.

The implementation helped demonstrate how Linux processes can exchange information, coordinate their activities, and use system calls for process communication.

---

## 17. Future Scope

The project can be extended to:

- Transfer larger amounts of data between processes.
  - Implement two-way communication.
  - Add multiple communicating processes.
  - Implement more advanced synchronization mechanisms.
  - Build a menu-driven IPC application.
-